

WHAT STRATEGY PATTERN IS:

It lets you switch algorithms/behaviors easily, like changing clothes.

Analogy: Video game character with different weapons:

Press 1 → Sword (close combat strategy)

Press 2 → Bow (ranged strategy)

Press 3 → Magic (spell strategy)

Same character, different attack strategies.

THE PROBLEM (What went wrong):

You made a navigation app (like Google Maps):

Version 1: Cars only

```
class Navigator:
    def build_route(self, start, end):
        # CAR route logic here
        return car_route
```

✅ Works fine! People driving cars love it.

Version 2: Add walking

Now people ask: "What about walking?"

```
class Navigator:
    def build_route(self, start, end, travel_type):
        if travel_type == "car":
            # 100 lines of car logic
        elif travel_type == "walk":
            # 100 lines of walking logic
```

😞 Getting bigger...

Version 3: Add public transport

```
class Navigator:
    def build_route(self, start, end, travel_type):
        if travel_type == "car":
            # 100 lines of car logic
        elif travel_type == "walk":
            # 100 lines of walking logic
        elif travel_type == "bus":
            # 100 lines of bus logic
```

😬 File is getting HUGE

Version 4: Add cycling

```
class Navigator:
    def build_route(self, start, end, travel_type):
        if travel_type == "car":
            # 100 lines
        elif travel_type == "walk":
            # 100 lines
        elif travel_type == "bus":
            # 100 lines
        elif travel_type == "bike":
            # 100 lines
```

📈 2000+ lines in one file!

🔥 THE DISASTER:

Bloat: One giant file with 2000+ lines

Team conflicts: 5 developers editing SAME file

Bugs: Fix car routing? Might break walking routing!

Adding new feature (like "scooter")? Scary!

This is like trying to fit ALL THESE in ONE SUITCASE:

Car 🚗

Bicycle 🚲

Bus 🚌

Walking shoes 👟

Scooter 🛴

Train 🚆

Everything else...

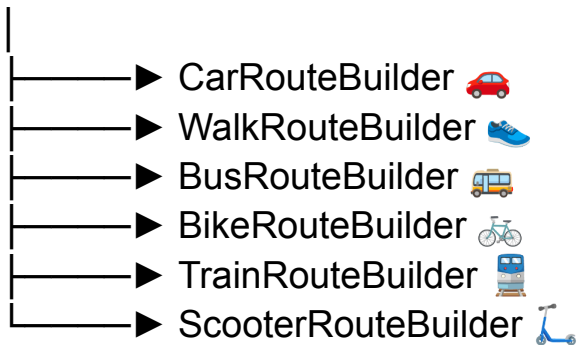
🎯 THE SOLUTION (Strategy Pattern):

Instead of ONE GIANT CLASS, create SEPARATE CLASSES:

Navigator (Your main app)



[RouteBuilder Interface]



HOW IT WORKS:

Before (Spaghetti Code):

ONE MESSY CLASS

```
class Navigator:
```

```
    def build_route(self, start, end, travel_type):
```

```
if travel_type == "car":
    # 200 lines car code
elif travel_type == "walk":
    # 200 lines walk code
elif travel_type == "bus":
    # 200 lines bus code
# ... 10 more types
```

After (Strategy Pattern):

```
# SEPARATE SMALL CLASSES
```

```
class CarRouteBuilder:
    def build(self, start, end):
        # Just car stuff here
        return car_route
```

```
class WalkRouteBuilder:
    def build(self, start, end):
        # Just walking stuff here
        return walk_route
```

```
class BusRouteBuilder:
    def build(self, start, end):
        # Just bus stuff here
        return bus_route
```

```
# MAIN APP (Clean!)
```

```
class Navigator:
    def __init__(self):
        self.route_builder = None

    def set_route_builder(self, builder):
        self.route_builder = builder

    def build_route(self, start, end):
        return self.route_builder.build(start, end)
```

USING IT IS EASY:

```
# Create navigator  
app = Navigator()
```

```
# User selects "Car"  
app.set_route_builder(CarRouteBuilder())  
route = app.build_route("Home", "Work")  
# Shows car route 🚗
```

```
# User changes to "Bike"  
app.set_route_builder(BikeRouteBuilder())  
route = app.build_route("Home", "Work")  
# Shows bike route 🚲
```

```
# Add NEW type (Scooter) LATER:  
class ScooterRouteBuilder: # Just add new class!  
    def build(self, start, end):  
        return scooter_route
```

```
app.set_route_builder(ScooterRouteBuilder())  
# Works instantly! 🛴
```

BENEFITS:

Small files: Car logic in car file, walk logic in walk file

Teamwork: Alice works on car, Bob works on bus - NO CONFLICTS

Safe changes: Fix bus bug? Can't break car code!

Easy to add: New "helicopter" route? Just add new class!

ANALOGY:

Imagine a pizza restaurant:

BAD WAY (One chef does everything):

Chef makes: Pepperoni pizza ✓

Chef makes: Veggie pizza ✓

Chef makes: Hawaiian pizza ✓

Chef makes: BBQ chicken pizza ✓

One chef knows 50 recipes → Overwhelmed!

GOOD WAY (Strategy Pattern):

Pepperoni Chef 🍳 (only knows pepperoni)

Veggie Chef 🍳 (only knows veggie)

Hawaiian Chef 🍳 (only knows hawaiian)

BBQ Chef 🍳 (only knows BBQ)

Manager (Navigator) assigns customer to right chef!

REAL APPS THAT USE THIS:

Google Maps: Switches between car/walk/bike/transit

Photo editors: Filters (blur, sharpen, grayscale)

Payment systems: Credit card / PayPal / Crypto

Compression tools: ZIP vs RAR vs 7Z algorithms



WITH STRATEGY PATTERN, YOU'LL LEARN:

1. RUNTIME FLEXIBILITY (Super Important!)
2. OPEN/CLOSED PRINCIPLE (SOLID Principle)
3. COMPOSITION OVER INHERITANCE
4. INTERFACE-BASED DESIGN
5. ALGORITHM ENCAPSULATION
6. CONDITIONAL LOGIC REFACTORING